Urban Mobility Data Explorer: Technical Report

Team Name: Cohort 1 **Team Members:** Favour Michael (Backend/Lead), Annabel Kemasuode (Data/DB), Hussaina Abubakar Shehu (Frontend)

1. Problem Framing and Dataset Analysis

Dataset Context: The application uses the official New York City Taxi & Limousine Commission (TLC) dataset, combining Yellow Taxi trip records (Fact table) with spatial and categorical metadata (Dimension tables). We specifically selected the January 2019 dataset to establish a robust, pre-pandemic baseline of urban mobility, avoiding the severe transit anomalies introduced by COVID-19 in later years. The objective is to decode urban movement patterns, examining transit economics, congestion metrics, and spatiotemporal flows across NYC boroughs.

Data Challenges & Cleaning Rules: Real-world transit data is notoriously noisy. We implemented strict cleaning rules and thresholds:

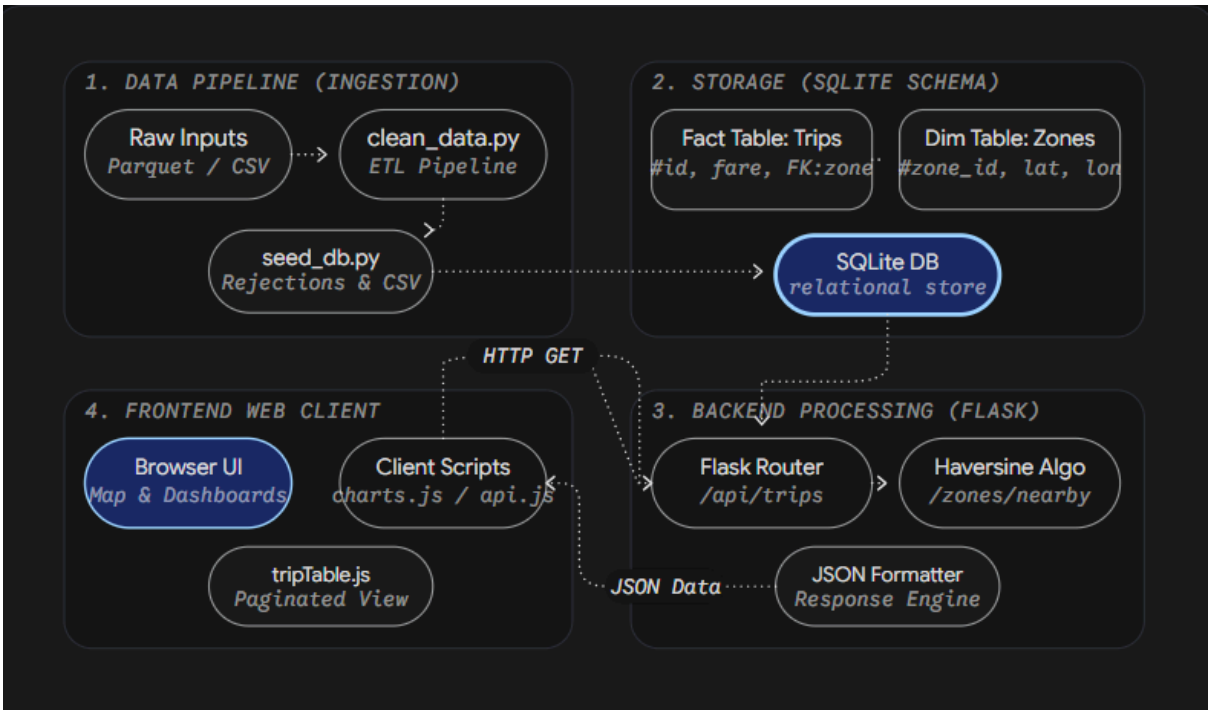
- **Distance & Duration:** Trips with a distance <= 0 miles or a duration <= 1 minute were filtered out as aborted trips or glitches.
- **Fare Anomalies:** Base fares <= $0 (likely refunds) and extreme outliers (fares > $500) were rejected.
- **Missing-Value Handling:** Rows with missing geographic coordinates, blank PULocationID, or missing DOLocationID were completely dropped rather than imputed, ensuring geographic accuracy.

Rejected-Row Counts: Our cleaning pipeline rejected a total of 113,189 rows. These were logged in a rejection log for transparency and data integrity tracking.

Derived Feature Formulas: To enhance our analytical dashboard, we engineered exactly three derived features:

1. **Average Speed (mph):** `trip_distance / duration_in_hours` (Identifies congestion bottlenecks)
2. **Cost Per Mile ($/mile):** `total_amount / trip_distance` (Reveals driver economic trade-offs)
3. **Trip Duration (Minutes):** `tpep_dropoff_datetime - tpep_pickup_datetime` (Standardizes temporal analysis)

2. System Architecture and Design Decisions



Stack Choices & Schema Normalization:

1. **Database (SQLite):** Selected for file-based portability. The schema is highly normalized: a `Trips` Fact table holding numeric measures, with foreign keys mapping to a `Zones` Dimension table (containing Borough names and spatial centroids).
2. **Indexes:** To ensure our backend queries run in milliseconds despite the large dataset, we created B-Tree indexes on the SQLite database specifically for `PULocationID`, `DOLocationID`, and `tpep_pickup_datetime`.
3. **Backend API Design (Python/Flask):** Chosen for rapid REST development and scientific math libraries. We exposed specific API endpoints: `GET /api/trips` (accepts query parameters for time, distance, and fare filtering) and `GET /api/zones/nearby` (accepts lat, lon, and radius for the custom spatial algorithm).
4. **Frontend (HTML/CSS/Vanilla JS):** We opted against heavy frameworks to minimize overhead. The dashboard uses native fetch() calls for state management and lightweight rendering.

Dataset Limitations & Trade-offs: By choosing SQLite, we sacrificed concurrent high-speed writes for development speed and system portability. A limitation of the dataset itself is the lack of exact GPS vehicle paths; we only have point-to-point (pickup/dropoff) zone mapping, which limits granular route-level analysis.

3. Algorithmic Logic and Data Structures

To enable users to filter transit patterns based on spatial proximity without relying on heavy spatial database extensions (like PostGIS), we manually implemented a radial distance algorithm on the backend.

The Problem: Users need to query trips that originated within a specific geographic radius (e.g., 3 kilometers) from a selected Taxi Zone centroid. Because the earth is spherical, standard Euclidean distance calculations yield inaccurate results.

The Implementation: We implemented the **Haversine Formula** manually in Python (spatial_utils.py) to calculate the great-circle distance between two sets of latitude and longitude coordinates. This function iterates over the dataset, returning only the zones that

fall within the user's defined radius.

Pseudo-Code:

```
FUNCTION calculate_haversine(lat1, lon1, lat2, lon2):
    SET R = 6371.0  // Earth's radius in km
    CONVERT all coordinates to radians
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = sin(dlat / 2)^2 + cos(lat1) * cos(lat2) * sin(dlon / 2)^2
    c = 2 * atan2(sqrt(a), sqrt(1 - a))
    RETURN R * c

FUNCTION filter_zones_by_radius(center_lat, center_lon, radius_km,
zone_list):
    INITIALIZE valid_zones = []
    FOR EACH zone IN zone_list:
        distance = calculate_haversine(center_lat, center_lon,
zone.lat, zone.lon)
        IF distance <= radius_km:
            APPEND zone TO valid_zones
    RETURN valid_zones
```

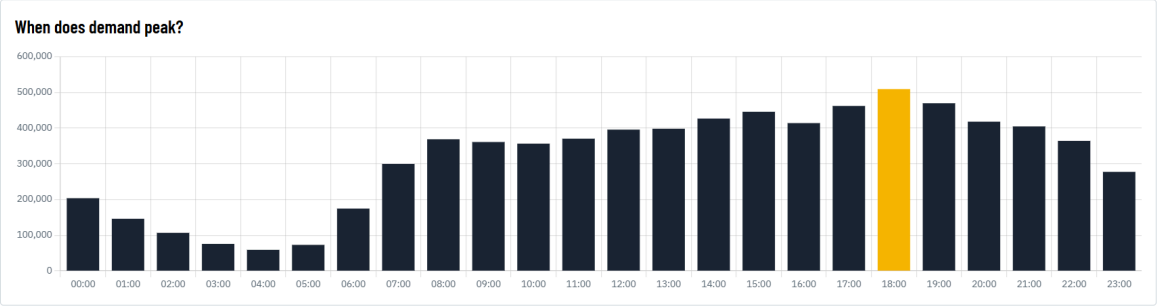
Complexity Analysis:

- **Time Complexity:** O(N), where N is the number of zones evaluated. The mathematical Haversine calculation resolves in constant time O(1), and the algorithm requires a single linear scan of the zone array.
- **Space Complexity:** O(K), where K is the number of zones that satisfy the radius condition and are stored in memory to be returned to the client.

4. Insights and Interpretation

Insight 1: The Evening Peak and Sustained Daily Demand

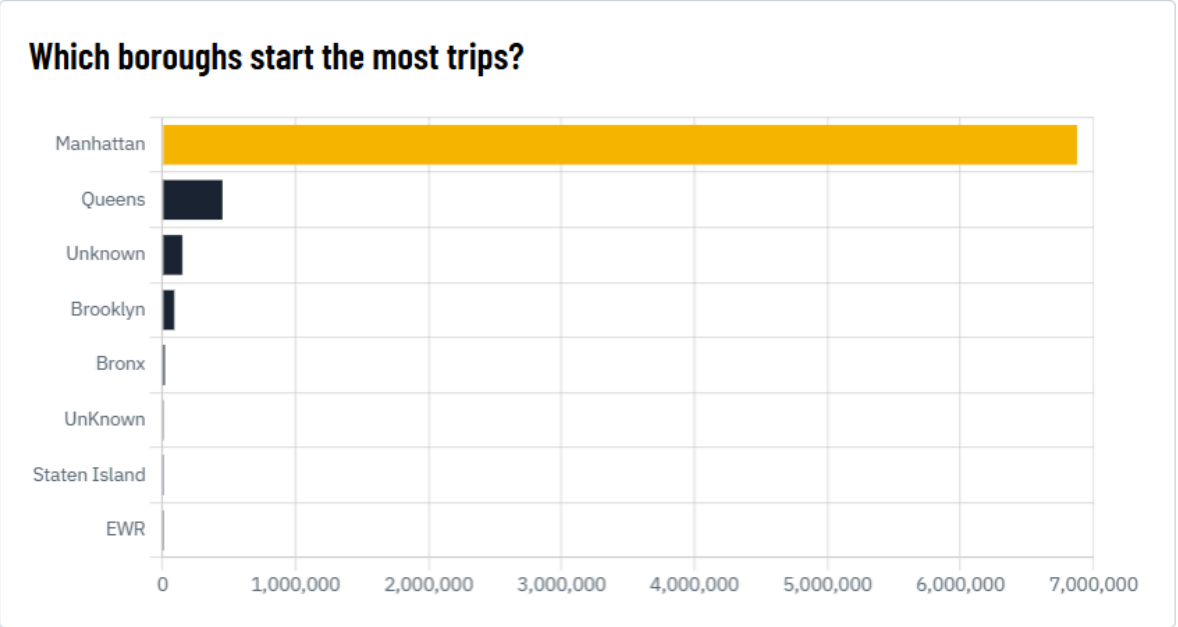
- **Derivation:** Grouping over 7.5 million trip volumes by hour of the day (00:00 to 23:00) using a vertical bar chart.



- **Interpretation:** Trip demand rises sharply at 07:00 for the morning commute and remains consistently high throughout the day. However, the absolute peak occurs at 18:00 (6:00 PM), suggesting a heavy overlap between workers leaving the office and the beginning of evening social/dining activities, before tapering off towards midnight.

Insight 2: Manhattan's Transit Dominance and Micro-Hubs

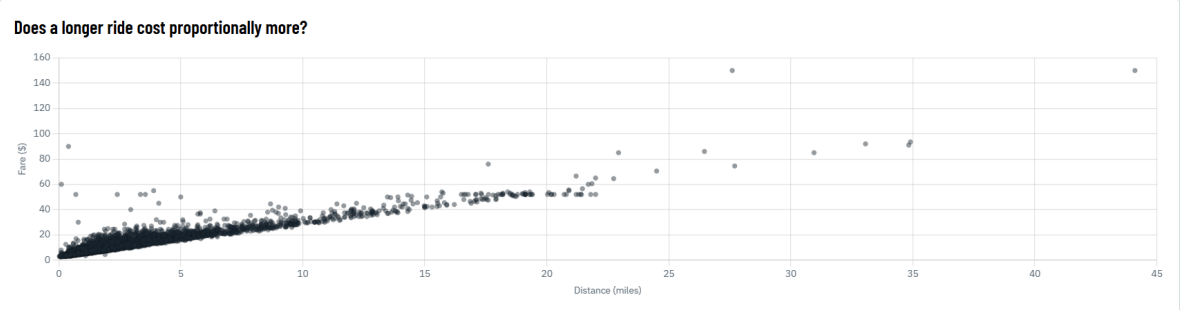
- **Derivation:** Categorizing total trip volumes by origin borough and specific neighborhood zones using horizontal bar charts.
- **Visual:**



- **Interpretation:** Manhattan accounts for the overwhelming majority of all yellow taxi trips, completely dwarfing outer boroughs like Queens and Brooklyn. Drilling down into specific micro-hubs, the busiest origin zones are highly concentrated in wealthy residential and commercial corridors, led sequentially by Upper East Side South, Midtown Center, and the Times Square/Theatre District.

Insight 3: The Short-Trip Density and Revenue Efficiency

- **Derivation:** Plotting individual trips on a scatter plot comparing Distance (miles) against Fare ($), alongside a granular data table.
- **Visual:**



- **Interpretation:** While a standard linear relationship exists between distance and cost, the scatter plot reveals a massive, dense clustering of trips under 5 miles and $20. The granular trip table shows the economic reality of this congestion: extremely short trips (e.g., 0.2 miles from Times Sq to Midtown North) travel at just 6.86 mph but yield a highly efficient $15 per mile for the driver, making dense, slow Manhattan corridors highly lucrative.

5. Reflection and Future Work

Challenges: The primary technical challenge was managing the sheer volume of the raw .parquet dataset on local machines. We overcame this by isolating the data-cleaning script from the API, generating a clean, indexed SQLite database that allowed the Flask server to run queries in milliseconds. Team coordination across the database, backend, and frontend required strict API contract adherence (defining exactly what JSON shape the frontend expected) to prevent bottlenecks.

Future Work: If deployed as a real-world product, the immediate next step would be migrating the database to PostgreSQL with the PostGIS extension. This would allow spatial indexing, reducing our geographic radius queries from O(N) linear time to O(\log N) logarithmic time. Additionally, implementing a Redis caching layer for the most commonly queried timeframes would significantly reduce database load during peak dashboard usage.

Frontend Challenges & API Contract Adherence: Maintaining high performance when rendering complex datasets required tight collaboration. By establishing a strict JSON API contract with the backend early on, we ensured data payloads were pre-filtered and aggregated before reaching the browser. On the UI side, proper chart instance lifecycle management (destroying old chart objects before rendering new ones) successfully prevented browser memory leaks during rapid user filter updates.